Applicant: Luis Valter Machado Junior

Full Project GitHub: https://github.com/AxeWalter/zipBoard-Test-Case

Documentation

This project is split in three different python scripts and one extra to "orchestrate the CRON job". They are: scraper.py, updates.py, gaps_analysis.py and main.py.

scraper.py

- The core logic of the project is in this file. It effectively crawls through all the articles available in https://help.zipboard.co/.
- It first opens all the categories in the website. After that, it opens each article in the category scraping for article name, category, URL, last updated, word count, checks for screenshots and video and extracts the text for latter analysis.
- After each scrape the URL is saved in a SET so it can avoid duplicates.
- For the content classification data, it uses a simple `if-elif` analyzing the title.
- For the ID a simple increment after each loop was used.
- The second part of this script is to enhance the data with the help of the Gemini API. The data is batched, using 40 articles in each submission, and the article id, title and text is feed to Gemini asking for 4 technical topics (`topics covered`) and for the individual gaps identified (`gaps identified`) in the articles. A JSON object is expected as return.
- The new information is added to the original data, enhancing the scraped data.
- The text column is removed from the data (it's not further necessary).
- This data is saved to a provided excel file path.

updated.py

- This script is responsible for updating the excel file if any of the defined changes are found.
- It first starts by doing a fresh scrape with the help of Scraper.py. This gathers fresh data to compare with the stored one.
- Loads the local data for comparison.
- If a new URL is detected or a change in the last updated happens, the script saves the rows affected.

- If no changes were detected, the script stops and prints a console message explaining that no changes were detected.
- If changes were detected, the script submits the new data to be enhanced through the same logic used in Scraper.py
- Saves the changed DF to excel

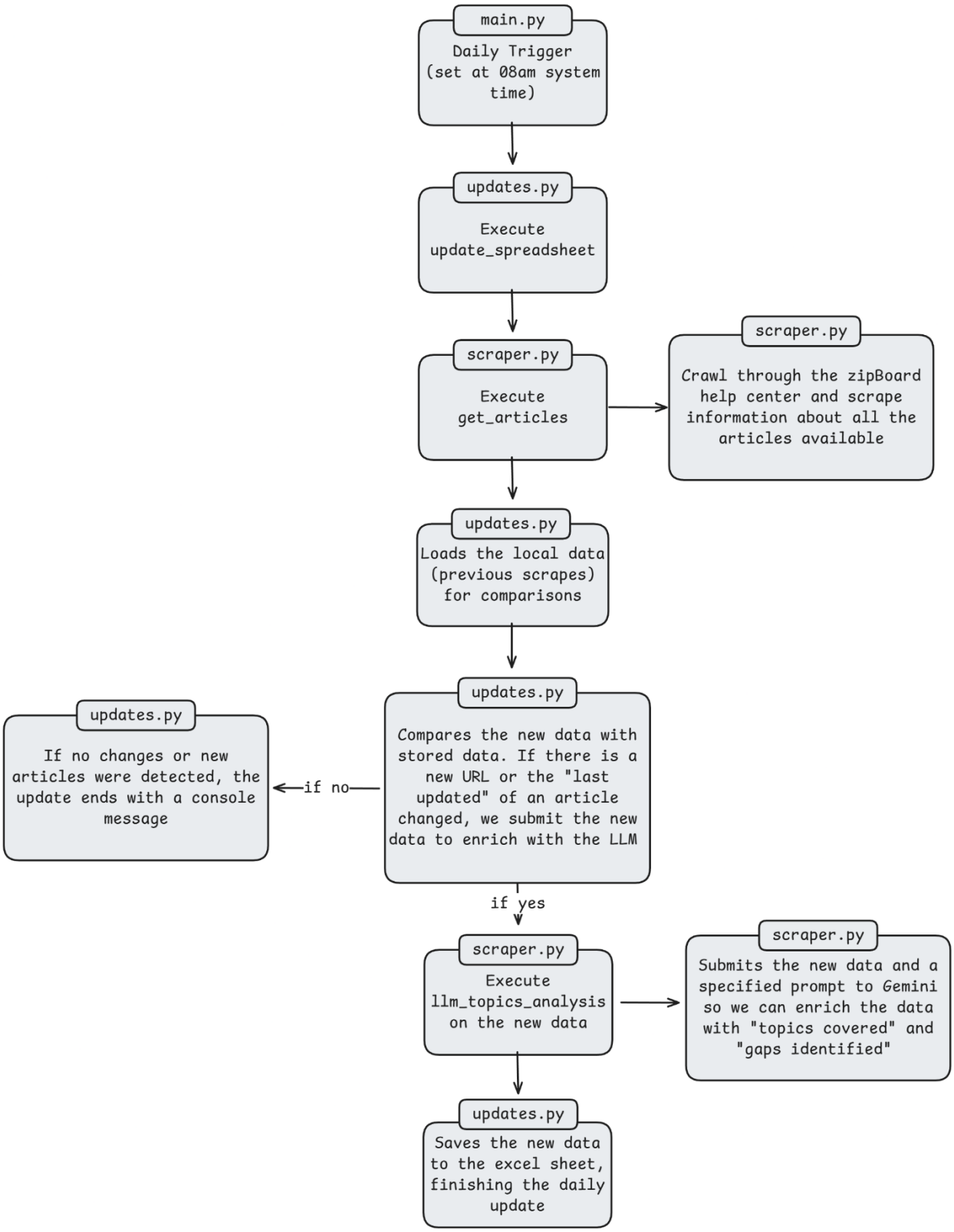
gaps_analysis.py

- This script is responsible for generating the top 10 gaps in the documentation and, using those 10 gaps, point the two most important and draw an article outline for them.
- It first loads the excel file so it can work on the data.
- Submits article id, category, article name and gaps identified, together with a prompt, to Gemini API asking for the top 10 global documentation gaps.
- Handles the expected Gemini JSON return and saves to an excel with the following structure: gap id, category, gap description, priority, suggested article title and rationale.
- The second part of this script is finding the 2 most important gaps in these 10. This is done by feeding Gemini the excel created in the last step together with a prompt. It expected that Gemini delivers the gap id, selection rationale and the article outline.
- The expected Gemini JSON is handled and saved to another sheet inside the top 10 global documentation gaps excel file.

main.py

- This is the script that allows the updated.py to be a “CRON job”.
- It simply uses the schedule python library to define a job that should run every day at 08am system time.
- It’ll run in the background checking every 60 seconds for the time.

Automation Workflow Diagram



Prompts Used

Scraper.py prompt for data enhancing:

Act as Senior Technical Writer and QA Auditor. For each article provided, perform two distinct tasks:

1. Extract 4 key technical topics (e.g. JIRA Cloud, JIRA Server, API, API Token, Webhooks, Mapping).

2. Identify ONE specific 'Documentation Gap'. We will define 'Documentation Gap' as a missing piece of critical information that a user would need based on the article's title (e.g. an article named 'API errors' not containing API error codes would have a gap that we can title 'Missing API error codes'). Base your analysis on good documentation practices for software and keep the 'Documentation Gap' brief, never exceeding 80 characters. If no gap is found, return 'None identified'."

Return ONLY a JSON object where the keys are the article ids, and the values are 'topics' and 'gap'. Example format: {'KB-001': {'topics': 'JIRA, Mapping', 'gap': 'Missing API token generation steps'}}

Gaps_analysis.py prompt for 10 global documentation gaps:

Act as Senior Technical Writer and QA Auditor. Your task is to analyze the provided data, access the provided competitor websites, analyze the competitor data and compare with the provided one to, in the end, return 10 Global Documentation Gaps in the zipBoard documentation. I'll be more specific by parts:

1. Context of what is zipBoard

zipBoard is a Design Review & task management tool that helps teams collaborate on digital content. It's a central hub for content and document reviews, allowing for multiple content & document type in one place.

2. Provided Data

I already scraped the whole help center from zipBoard. The important information is: the articles id, category, title and the individual gaps. This last information is important. We already ran analysis and have the individual gaps of each article (e.g. an article named 'API errors' not containing API error codes would have a gap that we can title 'Missing

API error codes'). You can use this information to infer global documentation gaps, like if a topic is missing in multiple articles it might be a global gap. But don't base yourself only on this information, use it together with the rest.
Below is the current information catalog:
{catalog_summary}

3.Competitor Websites
Compare the information found in this catalog with the information found in some competitors websites that provide a similar service. Think this as comparing to a possible golden standard. What documentation the competitor has that zipBoard does not? What services both provide but the competitor has a more in depth on how to use? The competitors are:
a. BugHerd (https://support.bugherd.com/en/);
b. MarkUp.io (https://educate.ceros.com/en/collections/15677333-faq);
c. Marker.io (https://help.marker.io/en/);
d. Usersnap (https://help.usersnap.com/).

4. Proper Description of task
Your task is to identify 10 Global Documentation Gaps in the zipBoard documentation. These are not about single articles, like the 'gap' column in the catalog, but looking at the zipBoard product and finding gaps in the documentations, things that could impact negatively the user, leaving them without answers. To this consider all the data provided (the context, catalog and competitor), and ponder about:
- Are there major features undocumented? (e.g. API, Security, specific integrations);
- Are there any major user questions that are not covered in the documentation?
- What documentation are present in the competitor’s website that is not available in the zipBoard?

5. Output format
Return a JSON object with a list called 'global_gaps'. Each gap must follow this EXACT table structure:
- Gap ID: (G-001 format);
- Category: (things like Integrations, Security, Onboarding, Developer, User, API, etc.);
- Gap Description: (1-2 sentences describing the gap);

- Priority: (High/Medium/Low);
- Suggested Article Title: (Catchy, helpful title);
- Rationale: (Why this gap matters for the user, why do we need to write an article about it)

Return ONLY the JSON object.

Gaps_analysis.py prompt for top 2 documentation gaps:

Act as Senior Technical Writer and QA Auditor. Your task is to analyze the provided data and consider the context of zipBoard to select the top 2 gaps present in the data, provide an explanation on why they are the most important gaps and write an article outline for those two gaps. I'll be more specific by parts:

1. Context of what is zipBoard

zipBoard is a Design Review & task management tool that helps teams collaborate on digital content. It's a central hub for content and document reviews, allowing for multiple content & document type in one place.

2. Provided Data

I already scraped the data from zipBoard's help center and analyzed it to the point of converging into the ten most important gaps in the documentation. This data I'll provide you contain the gap id, category, gap description, priority, suggested article title and a rationale for each individual gap. Use this information, together with the context and your knowledge about business, to fulfill the task.

Below is the data:

{gaps_summary}

3. Proper Description of task

a. Select, from the provided data, the TOP 2 gaps that have the highest priority and business impact (e.g. blocking sales, causing high churn, causing confusion to customers). It's IMPORTANT to note: select the two most important gaps, not simply the first two!

b. Provide a detailed explanation of why these two were chosen over the others and, most importantly, whey these are important to the customer;

c. Write a comprehensive article outline for each gap. The outline should include:

- Title of the article (use the one from the data provided);

- Goal of the article;
- Key sections (no need to write on them, just outline what should be disused);
- Brief introduction.

4. Output format

Return a JSON object with a list called 'deep_dive' containing exactly 2 objects.

Each object must have:

- Gap ID: (Match the ID from the list)
- Selection Rationale: (Why this is Top 2)
- Article Outline: (Detailed string with sections)

Return ONLY the JSON object.